
COA-II 实验报告——CPU 设计

GenuisL 2026/5

摘要

本项目设计了一款简单 CPU，仿照精简指令集（RISC）设计了本项目的指令集，采用微程序方式设计控制器，完成了整个 CPU 并自行设计了 4 个计算程序，通过了行为仿真、综合后时序仿真、布局布线后时序仿真和上板验证，确认设计的指令集全部工作正常。

本实验采用的开发板为 FPGA 数字信号处理板（CXD720），板载芯片型号为 XC7A100TFGG484-2，晶振频率 100MHz，本实验 CPU 主频即为晶振频率 100MHz，开发板上有 KEY1、KEY2、KEY3、KEY4、RST 共 5 个按钮，本实验中均设计了相应示例程序，板卡图片如下：

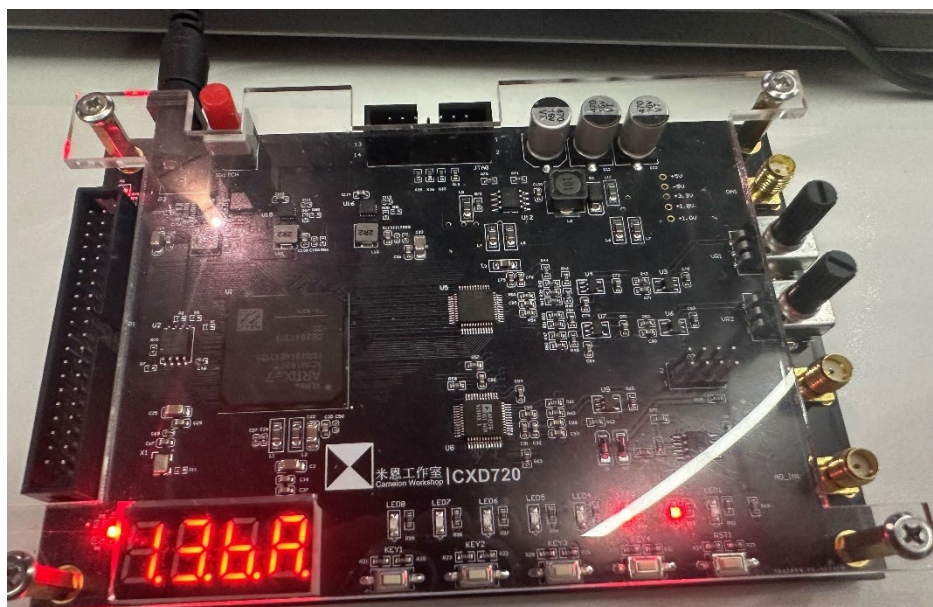


图 1 本实验使用的开发板

COA-II 实验报告——CPU 设计

目录

摘要.....	1
一、指令集.....	3
二、微指令.....	3
三、微指令、指令集、代码存储.....	6
1、微指令存储	6
2、微指令控制的指令集	7
3、运行代码	7
四、运行逻辑.....	10
1、顶层启动流程.....	10
2、CPU 何时真正开始执行	10
3、如何工作	10
五、仿真验证.....	13
1、行为仿真	13
2、综合后时序仿真	18
3、布局布线后时序仿真	19
六、上板测试.....	19
七、总结与反思.....	22
八、附录.....	24

一、指令集

为 CPU 设计了一个指令集，为简单起见，遵照课程示例，设置指令长度为 16 位，操作码和地址段长度如下：

OPCODE [15:8]	ADDRESS [7:0]
------------------	------------------

指令集如下表 1 所示：

INSTRUCTION	OPCODE	COMMENTS
LOAD X	00000001(01)	$[X] \rightarrow \text{ACC}$
STORE X	00000010(02)	$\text{ACC} \rightarrow [X]$
ADD X	00000011(03)	$\text{ACC} + [X] \rightarrow \text{ACC}$
SUB X	00000100(04)	$\text{ACC} - [X] \rightarrow \text{ACC}$
JMPGEZ X	00000101(05)	$\text{ACC} \geq 0 ? X \rightarrow \text{PC} : \text{PC} + 1 \rightarrow \text{PC}$
JMP X	00000110(06)	$X \rightarrow \text{PC}$
HALT	00000111(07)	Halt a program
MUL X	00001000(08)	$\text{ACC} * [X] \rightarrow \text{ACC}$
MUL X	00001001(09)	$\text{ACC} * [X] \rightarrow \text{MR}$
AND X	00001010(0A)	$\text{ACC} \text{ and } [X] \rightarrow \text{ACC}$
OR X	00001011(0B)	$\text{ACC} \text{ or } [X] \rightarrow \text{ACC}$
NOT X	00001100(0C)	$\text{NOT } [X] \rightarrow \text{ACC}$
SHIFTR X	00001101(0D)	SHIFT $[X]$ to Right 1bit, Logic Shift
SHIFTL X	00001110(0E)	SHIFT $[X]$ to Left 1bit, Logic Shift

表 1 指令集

二、微指令

为配套第一部分指令集运行，设计相应微指令控制表如下表 2 所示，其中 C0，即 $\text{CAR} \leq \text{CAR} + 1$ 仅在此处列出，实际程序中无需使用，直接用计数器控制

即可。

C0	$CAR \leq CAR+1$	Control Address Increment
C1	$CAR \leq ***$	Control Address Redirection depends on the position of microinstruction
C2	$CAR \leq 0$	Reset Control Address to zero position
C3	$MBR \leq \text{Memory}$	copy Memory content to MBR
C4	$IR \leq MBR[15:8]$	copy $MBR[15:8]$ to IR for opcode
C5	$MAR \leq MBR[7:0]$	copy $MBR[7:0]$ to MAR for address
C6	$PC \leq PC+1$	increment PC for indicating position
C7	$BR \leq MBR$	copy MBR value to BR
C8	$ACC \leq 0$	reset ACC register to zero
C9	$ACC \leq ACC+BR$	add BR to ACC
C10	$MAR \leq PC$	copy PC value to MAR for next address
C11	$ACC \leq BR$	copy BR value to ACC
C12	$MBR \leq ACC$	copy ACC value to MBR
C13	$\text{Memory} \leq MBR$	copy MBR value to Memory
C14	$ACC \leq ACC-BR$	ACC minus BR output stored in ACC
C15	$PC \leq MBR$	Memory Address Redirection
C16	$ACC, MR \leq ACC*BR$	ACC times BR output stored in ACC
C17	$ACC \leq ACC \& BR$	ACC AND BR output stored in ACC
C18	$ACC \leq ACC BR$	ACC OR BR output stored in ACC
C19	$ACC \leq \text{NOT}$	NOT BR output stored in ACC
C20	$ACC \leq (BR \gg 1)$	logic SHIFTR BR for one bit output stored in ACC
C21	$ACC \leq (BR \ll 1)$	logic SHIFTL BR for one bit output stored in ACC

表 2 微指令控制集

相应的微指令控制下的指令集如下表 3:

Instruction	Micro Operation	Control Signal	Location in ROM
FETCH	MAR<=PC	C10	rom[0]
	MBR<=Memory	C3	rom[1]
	IR<=MBR[15:8] MAR<=MBR[7:0] PC<=PC+1	C4 C5 C6	rom[2]
	CAR<=***	C1	rom[3]
LOAD X	MBR<=Memory	C3	rom[10]
	BR<=MBR	C7	rom[11]
	ACC<=BR CAR<=0	C11 C2	rom[12]
STORE X	MBR<=ACC	C12	rom[20]
	Memory<=MBR	C13	rom[21]
	CAR<=0	C2	rom[22]
ADD X	MBR<=Memory	C3	rom[30]
	BR<=MBR	C7	rom[31]
	ACC<=ACC+BR CAR<=0	C9 C2	rom[32]
SUB X	MBR<=Memory	C3	rom[40]
	BR<=MBR	C7	rom[41]
	ACC<=ACC-BR CAR<=0	C14 C2	rom[42]
JMPGEZ X	PC<=MBR[7:0] CAR<=0	C15 C2	rom[50]
JMP X	PC<=MBR[7:0] CAR<=0	C15 C2	rom[60]
HALT	CAR<=0	C2	rom[70]
MUL X	MBR<=Memory	C3	rom[80]

	BR<=MBR	C7	rom[81]
	ACC<=ACC*BR CAR<=0	C16 C2	rom[82]
AND X	MBR<=Memory	C3	rom[90]
	BR<=MBR	C7	rom[91]
	ACC<=ACC&BR CAR<=0	C17 C2	rom[92]
OR X	MBR<=Memory	C3	rom[100]
	BR<=MBR	C7	rom[101]
	ACC<=ACC BR CAR<=0	C18 C2	rom[102]
NOT X	MBR<=Memory	C3	rom[110]
	BR<=MBR	C7	rom[111]
	ACC<=~BR CAR<=0	C19 C2	rom[112]
SHIFTR X	MBR<=Memory	C3	rom[120]
	BR<=MBR	C7	rom[121]
	ACC<=BR>>1 CAR<=0	C20 C2	rom[122]
SHIFTL X	MBR<=Memory	C3	rom[125]
	BR<=MBR	C7	rom[126]
	ACC<=BR<<1 CAR<=0	C21 C2	rom[127]

表 3 指令集的微指令控制表

三、微指令、指令集、代码存储

1、微指令存储

微指令定义了周期会做的事情，为了 CPU 完成某个指令，需要在若干个微操作周期内完成若干的微操作。本实验所用到的所有微指令和指令集均存储于 ControlMemory.v。其中，微指令的存储方式如下，通过 localparam 来定义微指令，好处是在下方定义微指令控制的指令集时，能更更加直观的看到每个单独的指令是如何设计微指令的：

```

localparam [21:0] C1  = (22'b1 << 1);
localparam [21:0] C2  = (22'b1 << 2);
localparam [21:0] C3  = (22'b1 << 3);
localparam [21:0] C4  = (22'b1 << 4);
localparam [21:0] C5  = (22'b1 << 5);
localparam [21:0] C6  = (22'b1 << 6);
localparam [21:0] C7  = (22'b1 << 7);
localparam [21:0] C9  = (22'b1 << 9);
localparam [21:0] C10 = (22'b1 << 10);
localparam [21:0] C11 = (22'b1 << 11);
localparam [21:0] C12 = (22'b1 << 12);
localparam [21:0] C13 = (22'b1 << 13);
localparam [21:0] C14 = (22'b1 << 14);
localparam [21:0] C15 = (22'b1 << 15);
localparam [21:0] C16 = (22'b1 << 16);
localparam [21:0] C17 = (22'b1 << 17);
localparam [21:0] C18 = (22'b1 << 18);
localparam [21:0] C19 = (22'b1 << 19);
localparam [21:0] C20 = (22'b1 << 20);
localparam [21:0] C21 = (22'b1 << 21);

```

由上可见,本设计里 C0 没有单独在 rom 中使用,这是由于指令集比较简单, CAR 只需默认在 CU 中按顺序加 1 即可,前面表 2 列出 C0 仅仅是为了微指令集的完备性。

2、微指令控制的指令集

指令集储于 ControlMemory.v 的 rom 寄存器中,该 rom 设置为 $128 \times 22\text{bit}$,其中 22bit 为微指令长度,128 为储存深度。这里无需调用 Vivado 提供的 ROM IP 核,因为该存储寄存器矩阵并不大,手动编写反而易于管理。如下列出举例 ADD X 的微指令,详细微指令表参见表 3 以及 ControlMemory.v

```

// ----- ADD X : opcode = 03 -----
// 目标: ACC<=ACC+[X]
rom[30] = C3;           // MBR<=Memory    ; 读出地址 X 中的数据
rom[31] = C7;           // BR<=MBR        ; 数据送 BR
rom[32] = C9 | C2;      // ACC<=ACC+BR,CAR<=0

```

3、运行代码

运行代码存储于 InstructionMemory.v 中的 mem 中,其大小为 $256 \times 16\text{bit}$,即操作码长度为 16bit,储存深度为 256。操作码在表 1 前已列出,设计以下四个运算的代码来验证 CPU 设计正确,每次按下测试按键时,都会重新装载对应的程序和数据镜像,因此不会受到上一次 STORE 改写数据区的影响,适合重复上

板测试。

A. 累加代码公式如式 1 所示，正确结果 5050(0x13BA)

$$ACC = \sum_{i=1}^{100} i \quad (1)$$

相应指令代码：

```
// key[1] : 1+2+...+100 = 5050=0x13BA 综合测试，正确结果 5050(0x13BA)
mem[8'h00] = 16'h010B; // LOAD 0x0B -> ACC = sum, initial value 0
mem[8'h01] = 16'h030C; // ADD 0x0C -> ACC = sum + i
mem[8'h02] = 16'h020B; // STORE 0x0B -> sum = ACC
mem[8'h03] = 16'h010C; // LOAD 0x0C -> ACC = i
mem[8'h04] = 16'h030D; // ADD 0x0D -> ACC = i + 1
mem[8'h05] = 16'h020C; // STORE 0x0C -> i = i + 1
mem[8'h06] = 16'h040E; // SUB 0x0E -> ACC = i - 101
mem[8'h07] = 16'h0509; // JMPGEZ 0x09 -> if i >= 101 then jump to output
mem[8'h08] = 16'h0600; // JMP 0x00 -> loop back
mem[8'h09] = 16'h010B; // LOAD 0x0B -> ACC = final sum
mem[8'h0A] = 16'h0700; // HALT -> stop, ACC should be 5050
mem[8'h0B] = 16'h0000; // data sum = 0
mem[8'h0C] = 16'h0001; // data i = 1
mem[8'h0D] = 16'h0001; // data one = 1
mem[8'h0E] = 16'h0065; // data 101 = 101, used by SUB for loop end
```

计算案例 1 1 至 100 累加代码

B. MUL / AND / OR 综合测试，正确结果 287(0x011F)

相应指令代码：

```
// key[2] : MUL / AND / OR 综合测试，正确结果 287(0x011F)
mem[8'h00] = 16'h0110; // LOAD 0x10 -> 6
mem[8'h01] = 16'h0811; // MUL 0x11 -> 6*7=42
mem[8'h02] = 16'h0215; // STORE 0x15
mem[8'h03] = 16'h0112; // LOAD 0x12 -> 0x00F0
mem[8'h04] = 16'h0A13; // AND 0x13 -> 0x00F0
mem[8'h05] = 16'h0B14; // OR 0x14 -> 0x00F5
mem[8'h06] = 16'h0315; // ADD 0x15 -> 0x00F5 + 42 = 287
mem[8'h07] = 16'h0700; // HALT
mem[8'h10] = 16'h0006; // data A = 6
mem[8'h11] = 16'h0007; // data B = 7
mem[8'h12] = 16'h00F0; // data C = 0x00F0
mem[8'h13] = 16'h0FF0; // data D = 0x0FF0
mem[8'h14] = 16'h0005; // data E = 5
```

```
mem[8'h15] = 16'h0000; // data TEMP = 0, stores MUL result 42
```

计算案例 2 MUL / AND / OR 综合测试

C. NOT / SHIFTR / SHIFTL 综合测试, 正确结果 65310(0xFF1E)

```
// key[3] : NOT / SHIFTR / SHIFTL 综合测试, 正确结果 65310(0xFF1E)
mem[8'h00] = 16'h0C10; // NOT    0x10 -> ~0x00F0 = 0xFF0F
mem[8'h01] = 16'h0213; // STORE  0x13
mem[8'h02] = 16'h0D11; // SHIFTR 0x11 -> 0x0009
mem[8'h03] = 16'h0214; // STORE  0x14
mem[8'h04] = 16'h0E12; // SHIFTL 0x12 -> 0x0006
mem[8'h05] = 16'h0314; // ADD     0x14 -> 0x000F
mem[8'h06] = 16'h0313; // ADD     0x13 -> 0xFF1E
mem[8'h07] = 16'h0700; // HALT
mem[8'h10] = 16'h00F0; // data A = 0x00F0, used by NOT
mem[8'h11] = 16'h0012; // data B = 0x0012, used by SHIFTR -> 0x0009
mem[8'h12] = 16'h0003; // data C = 0x0003, used by SHIFTL -> 0x0006
mem[8'h13] = 16'h0000; // data TEMP1 = 0, stores NOT result
mem[8'h14] = 16'h0000; // data TEMP2 = 0, stores SHIFTR result
```

计算案例 3 NOT / SHIFTR / SHIFTL 综合测试

D. JMPGEZ / JMP 分支测试, 正确结果 1234(0x04D2)

```
// key[4] : JMPGEZ / JMP 分支测试, 正确结果 1234(0x04D2)
mem[8'h00] = 16'h0110; // LOAD   0x10 -> 1
mem[8'h01] = 16'h0504; // JMPGEZ 0x04, 条件成立, 应跳转
mem[8'h02] = 16'h0115; // LOAD   0x15, 错误路径, 应被跳过
mem[8'h03] = 16'h0700; // HALT, 错误路径, 应被跳过
mem[8'h04] = 16'h0111; // LOAD   0x11 -> 0
mem[8'h05] = 16'h0412; // SUB     0x12 -> -1
mem[8'h06] = 16'h0508; // JMPGEZ 0x08, 条件不成立, 不应跳转
mem[8'h07] = 16'h0609; // JMP     0x09, 跳到正确出口
mem[8'h08] = 16'h0116; // LOAD   0x16, 错误路径, 不应执行
mem[8'h09] = 16'h0113; // LOAD   0x13 -> 1234
mem[8'h0A] = 16'h0700; // HALT
mem[8'h10] = 16'h0001; // data A = 1
mem[8'h11] = 16'h0000; // data B = 0
mem[8'h12] = 16'h0001; // data C = 1, used to create -1 by SUB
mem[8'h13] = 16'h04D2; // data RESULT = 1234, correct final answer
mem[8'h15] = 16'h1111; // data WRONG1 = 0x1111, should never be loaded finally
mem[8'h16] = 16'h2222; // data WRONG2 = 0x2222, should never be loaded finally
```

计算案例 4 JMPGEZ / JMP 分支测试

四、运行逻辑

1、顶层启动流程

顶层模块位于 `sources_1/new/CpuTop.v`，它把时钟、按键、CPU 数据通路、控制器、程序存储器和数码管显示全部连接在一起。整个工程不是上电后自动连续执行，而是采用“按键触发一次完整运行”的方式。

运行入口有两类：

- 按下 `rst`：产生一次复位脉冲，同时把当前程序选择恢复为默认的 `program 1`
- 按下 `key[1]`到`key[4]`：选择对应的测试程序，并立即触发一次新的运行

在顶层里 `key1_pulse | key2_pulse | key3_pulse | key4_pulse` 会合成为 `key_run_pulse`。随后

```
cpu_rst = key_reset_pulse | key_run_pulse
```

```
load_program = key_reset_pulse | key_run_pulse
```

这意味着无论是按复位键，还是按测试键，都会同时发生两件事：

- i. CPU 内部寄存器被复位
- ii. 程序存储器重新装载一份干净的测试镜像

这样设计的好处是，前一次运行中 `STORE` 写回到内存的数据，不会污染下一次实验结果，所以每次重新运行都是从确定的初始状态开始。

2、CPU 何时真正开始执行

顶层中的 `cpu_run` 是整个 CPU 的运行许可位，按下某个程序选择键时，`cpu_run` 被置为 1；当检测到 `HALT` 指令时，`cpu_run` 被清零真正推动 CPU 前进的使能信号是 `cpu_step_en = cpu_run & ~halt_flag`

只有当 CPU 处于运行状态且当前指令不是 `HALT` 时，寄存器和控制地址寄存器 `CAR` 才会在时钟上升沿更新。

3、如何工作

本实验仿照了传统冯诺依曼架构下的 `RSIC-V` 设计，简单的逻辑图如下所示

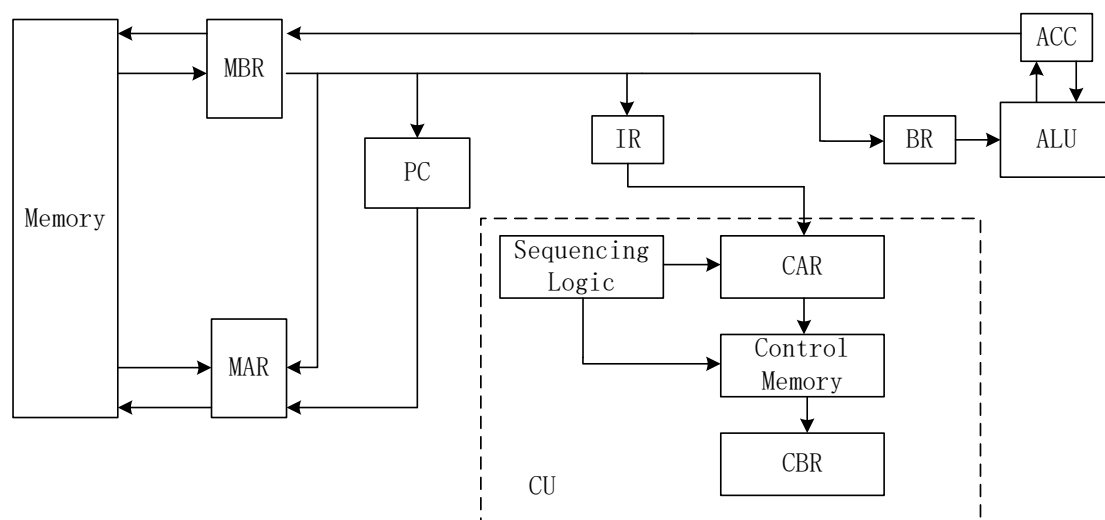


图 2 本实验的 CPU 架构图

整个运行流程与 COA-I 课上讲的运行流程相同，因此下面简要说明本实验的大致运行流程。

首先是 Instruction Cycle，在运行每行指令时，都需要经过取指、执行的过程即经典的 Instruction Cycle

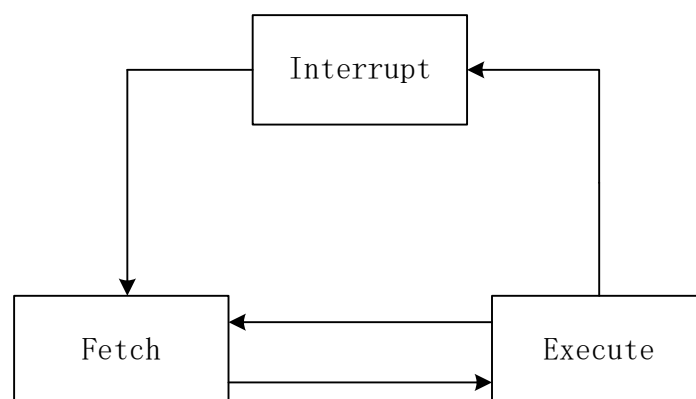


图 3 Instruction Cycle

我们结合行为仿真图来说明

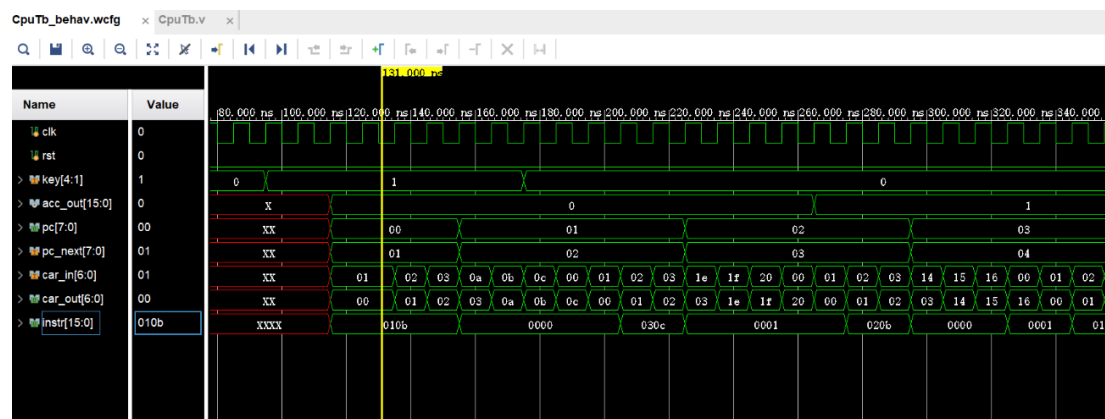


图 4 行为仿真图

我们着重看 pc[7:0]和 car_out[6:0]，可以看到，当 PC 变化时若执行的不是中断，则 PC 所覆盖的时钟段会分为两部分，一部分是 CAR 为 00、01、02、03，照应到表 3 即为 FETCH，取到了相应的指令后，再进行 Execute。

取值周期经历了

1. 把程序计数器 PC 指向的地址送到 MAR
2. 读出该地址对应的 16 位机器指令到 'MBR'
3. 把高 8 位操作码送入 IR，把低 8 位地址字段送入 MAR，同时 PC 自增
4. 控制器根据操作码决定接下来进入哪一段执行微程序

进入 Control Unit 中，控制器由 CU.v + CAR.v + ControlMemory.v 共同完成。

CAR 保存当前微地址，ControlMemory 根据 CAR 读出 22 位微指令，CU 根据当前微指令内容、机器指令操作码以及 ACC 的符号，生成下一拍的 next_car，CU.v 中下一微地址的规则可以总结为三种：

1. 如果微指令要求 $CAR \leq 0$ ，就回到公共取指入口
2. 如果微指令要求按操作码分支，就根据 'IR' 里的操作码跳到某条机器指令的微程序入口
3. 否则 $CAR = CAR + 1$ ，顺序执行下一条微指令

其中 JMPGEZ 比较特殊，它不是简单跳到固定入口，而是先看 ACC[15]：

- ACC[15] = 0：认为 $ACC \geq 0$ ，进入跳转执行入口
- ACC[15] = 1：认为 $ACC < 0$ ，直接回到取指周期，相当于条件不满足、不跳转

五、仿真实验

1、行为仿真

设置仿真显示的值，较为重要的有：CLK、CAR、PC、ACC 等，进行仿真得到下面的仿真结果，首先我们观察 CAR 和 PC 的运行情况，如下图

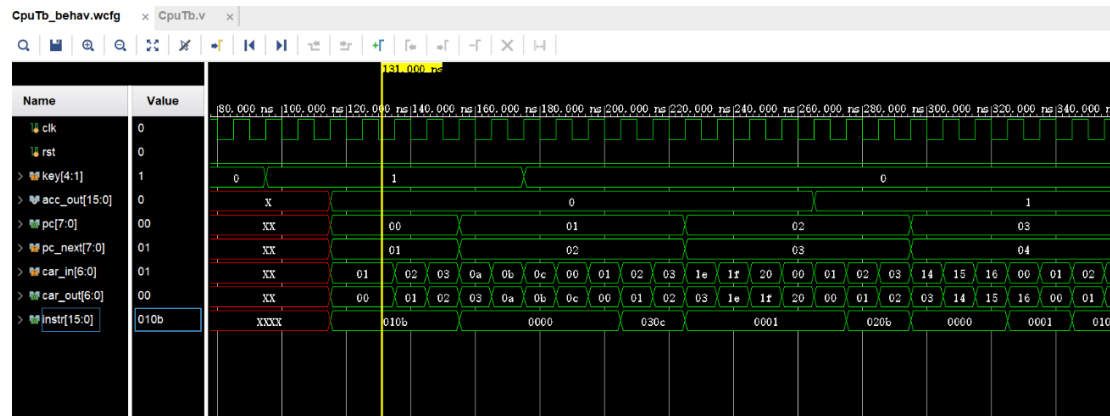


图 5 行为仿真的 CAR、PC

细看图片可知，在运行过程中，CAR 控制的 car_in、car_out 相差一个时钟周期，这是由于 car_in 是在 car_out 的基础上默认加 1 或指向下一个指令，可见，在跳转时即 PC 变化后的下一个时钟周期，car 分别指向了 00（FETCH），0a（LOAD），00（FETCH），1e（ADD），00（FETCH），14（STORE）……查看计算案例 1 可知，该 CAR 正是指向了每个指令的第一个微指令，因此可以判断 PC 与 CAR 均运行正确。

下面再检查程序逻辑是否运行正确，根据计算案例 1 可知，内存 mem[8'h0B] 存放了计算后累加的结果，，即在每个 STORE 指令后，都把 ACC 的值存放在了该内存中，我们只需要观察 ACC 的运行结果即可，如下图所示

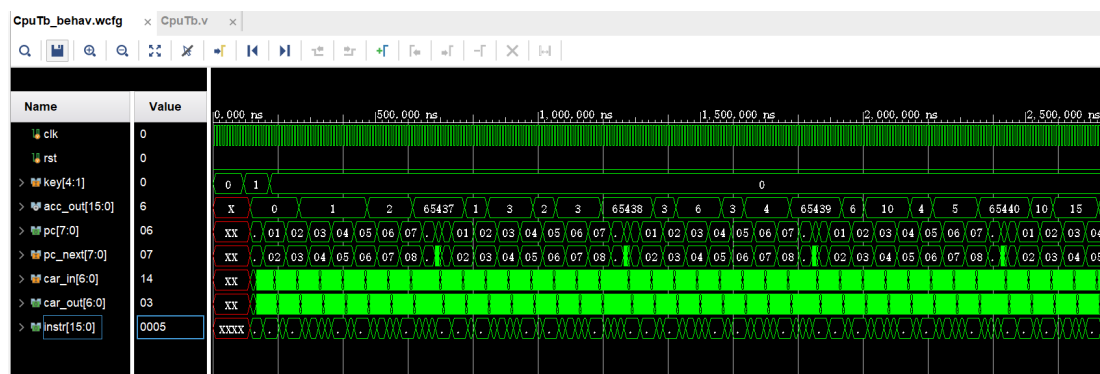


图 6 运行逻辑检查（示例程序 1 开始）

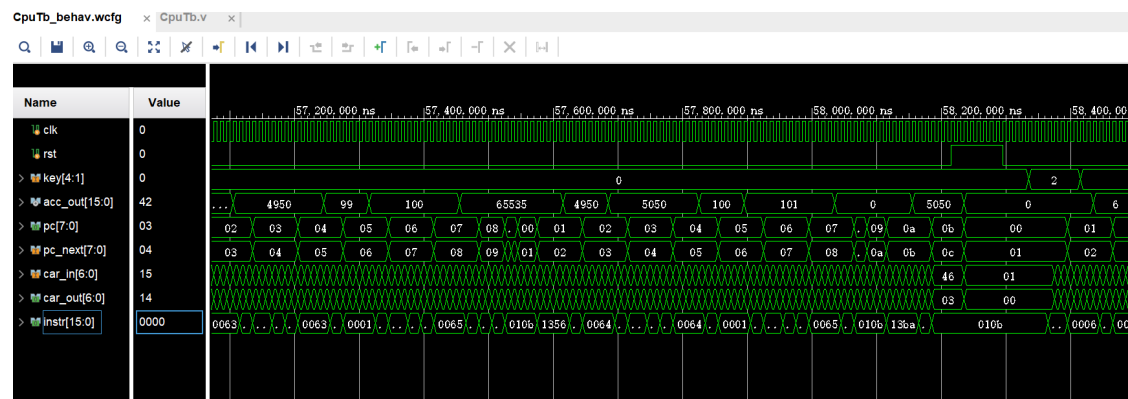


图 7 运行逻辑检查（示例程序 1 结束）

我们根据 PC 的运行来检查 ACC 是否每一步都运行正确。已知内存中存储变量为

```
mem[8'h0B] = 16'h0000; // data sum  = 0
mem[8'h0C] = 16'h0001; // data i    = 1
mem[8'h0D] = 16'h0001; // data one  = 1
mem[8'h0E] = 16'h0065; // data 101  = 101, used by SUB for loop end
```

当模拟按下 KEY1 后，即 key[4:1]变为 1 后，开始运行示例程序 1，如下过程：

PC	指令	操作	ACC（实际）	备注
00	LOAD 0x0B	ACC=sum	0	读取 0B 的值
01	ADD 0x0C	ACC = sum + i	1	
02	STORE 0x0B	sum = ACC	1	
03	LOAD 0x0C	ACC = i	1	
04	ADD 0x0D	ACC = i + 1	2	
05	STORE 0x0C	i = i + 1	2	
06	SUB 0x0E	ACC = i - 101	-99（65437）	补码运算
07	JMPGEZ 0x09	if i >= 101 then jump to output	-99（65437）	不满足，跳过
08	JMP 0x00	loop back	-99（65437）	回到 00
00	LOAD 0x0B	ACC = sum	1	下一个循环

..... 循环到最后一个周期				
00	LOAD 0x0B	ACC=sum	4950	
01	ADD 0x0C	ACC = sum + i	5050	
02	STORE 0x0B	sum = ACC	5050	
03	LOAD 0x0C	ACC = i	100	
04	ADD 0x0D	ACC = i + 1	101	
05	STORE 0x0C	i = i + 1	101	
06	SUB 0x0E	ACC = i - 101	0	
07	JMPGEZ 0x09	if i >= 101 then jump to output	0	满足条件
08	JMP 0x00	loop back	0	跳过
09	LOAD 0x0B	ACC = sum	5050	
0A	HALT	stop	5050	结果正确

表 4 运行示例程序 1 过程

由上表 4 可见，当循环结束后，ACC 的值锁定在了 5050，程序 HALT 挂起，可见该示例程序运行正确。

同理我们运行分析其他示例程序如下

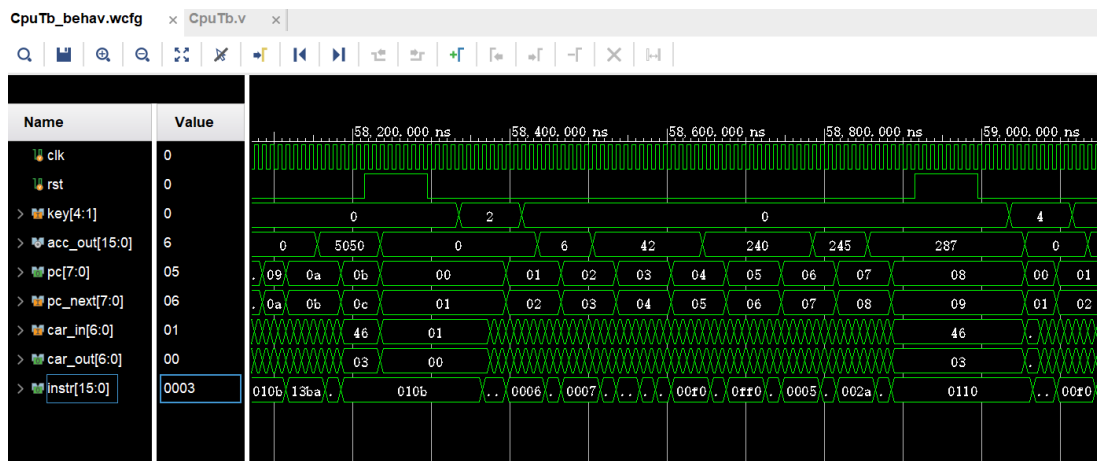


图 8 运行逻辑检查（示例程序 2）

内存变量如下

```

mem[8'h10] = 16'h0006; // data A = 6
mem[8'h11] = 16'h0007; // data B = 7
mem[8'h12] = 16'h00F0; // data C = 0x00F0
mem[8'h13] = 16'h0FF0; // data D = 0x0FF0
mem[8'h14] = 16'h0005; // data E = 5
mem[8'h15] = 16'h0000; // data TEMP = 0, stores MUL result 42

```

运行逻辑如下

PC	指令	操作	ACC（实际）	备注
00	LOAD 0x10	提取 data A	6	
01	MUL 0x11	$6 \times 7 = 42$	42	
02	STORE 0x15	存放 TEMP	42	
03	LOAD 0x12	提取 data C	0xF0	240
04	AND 0x13	$\text{acc} \& 0x0FF0$	0x00F5	245
05	OR 0x14	$\text{acc} 5$	0x00F5	245
06	ADD 0x15	$0x00F5 + 42$	287	
07	HALT			暂停

表 5 运行示例程序 2 过程

可见，示例程序 2 也运行正确，下面示例程序 3 和示例程序 4 原理与上相同，这里仅列出结果展示：

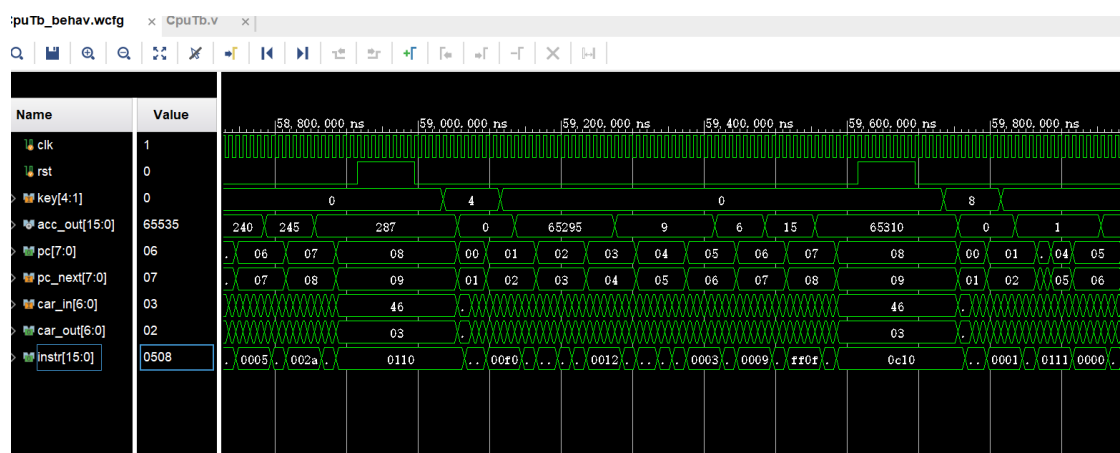


图 9 运行逻辑检查（示例程序 3）

结果为 65310(0xFF1E), 结果正确

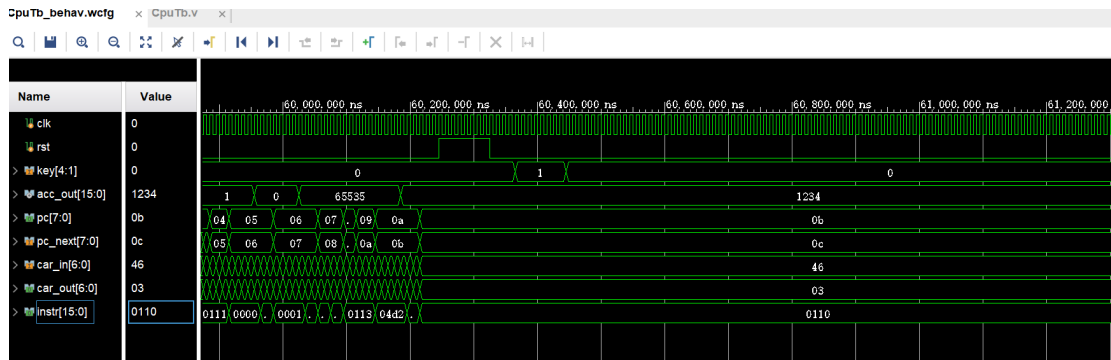


图 10 运行逻辑检查（示例程序 4）

结果为 1234(0x04D2), 结果正确

同时, 为了方便观察仿真结果, 在 tb 文件中编写判断逻辑并输出到控制台, 示例如下, 详见附录:

```
if (led[2] != 1'b1) begin
    $display("=====");
    $display("TEST FAIL : %0s", name);
    $display("REASON      : TIMEOUT WAITING FOR HALT");
    $display("CYCLES       : %0d", cycle_count);
    $display("=====");
    $finish;
end
repeat (2) @(posedge clk);
if (uut.acc_out == expected) begin
    $display("=====");
    $display("TEST PASS : %0s", name);
    $display("ACC        : %0d (0x%04h)", uut.acc_out, uut.acc_out);
    $display("=====");
end else begin
    $display("=====");
    $display("TEST FAIL : %0s", name);
    $display("EXPECTED   : %0d (0x%04h)", expected, expected);
    $display("ACTUAL     : %0d (0x%04h)", uut.acc_out, uut.acc_out);
    $display("=====");
    $finish;
end
end
```

每当运行完一个程序后程序自动判断是否输出正确, 并将结果反馈到控制台。

如下图所示

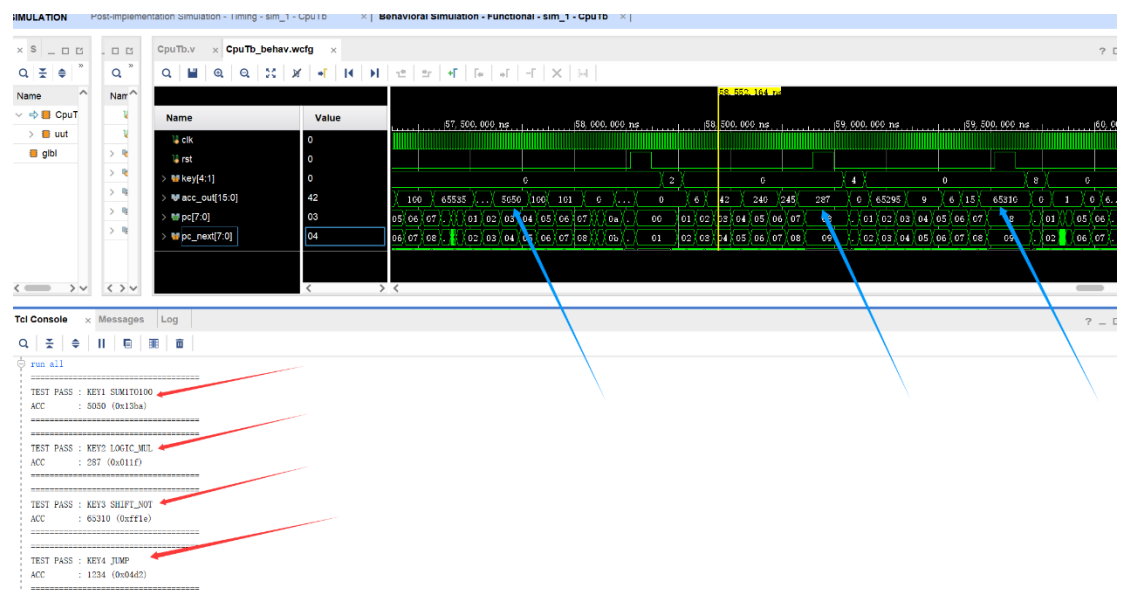


图 11 判断正误并输出到控制台

可见，程序自动判断结果也是正确的

2、综合后时序仿真

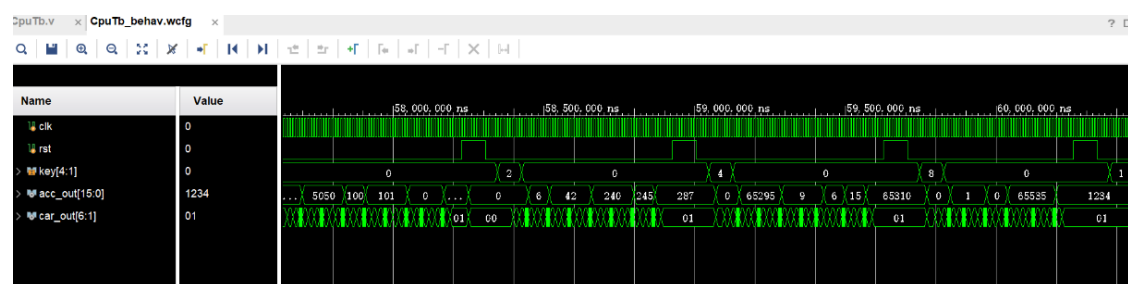


图 12 综合后时序仿真

从图中可总结下表

运行程序	综合后时序仿真结果	理想结果	判断
1	5050	5050	正确
2	287	287	正确
3	65310	65310	正确
4	1234	1234	正确

表 6 综合后时序仿真结果判断

由表 6 可知，该综合后时序仿真结果正确

3、布局布线后时序仿真

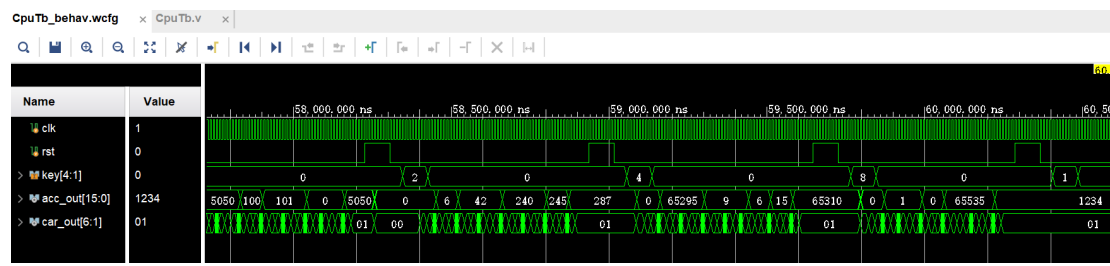


图 13 布局布线后时序仿真

从图中可总结下表

运行程序	综合后时序仿真结果	理想结果	判断
1	5050	5050	正确
2	287	287	正确
3	65310	65310	正确
4	1234	1234	正确

表 7 布局布线后时序仿真

由表 7 可知，该布局布线后时序仿真结果正确。

据此，完成了行为仿真、综合后时序仿真、布局布线后时序仿真，下面可以进行上板测试环节。

六、上板测试

本实验将生成的 bit 流文件固化到了开发板的 FLASH 当中，本开发板的 FLASH 大小为 128M bits，因此在 Vivado 选择生成时，要将 FLASH 大小设置为 16M Bytes。生成.mcs 文件放到工程目录下方，同步会生成.prm 文件，在烧录固化到开发板时正确选择即可。

烧录到 FLASH 的好处是固化后断电开发板程序不会因为断电而消失，因此无需再连接烧录器至电脑。

下面演示结果，其中 LED 的信息为：从右至左为 LED1 至 LED8，LED2 亮表示当前程序运行完成且正确，LED3 至 LED6 亮分别对应表示正在运行程序 1 至程序 4。默认预载程序为程序 1，即公式（1）。

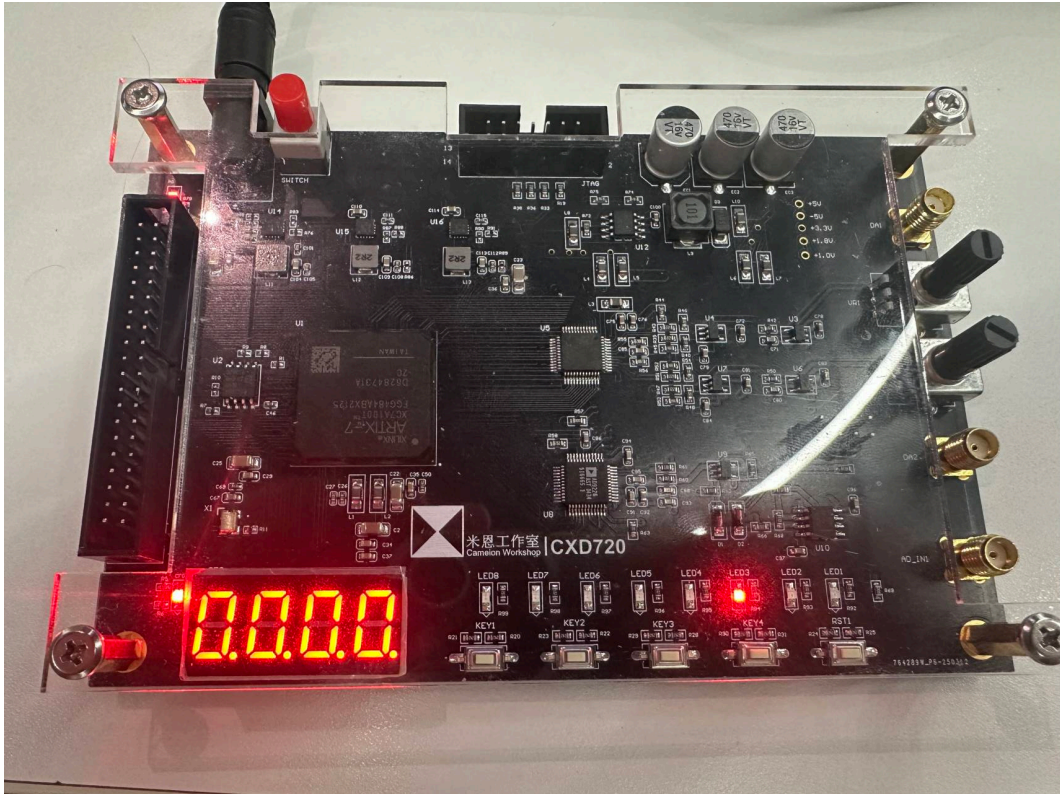


图 14 按下 RST1

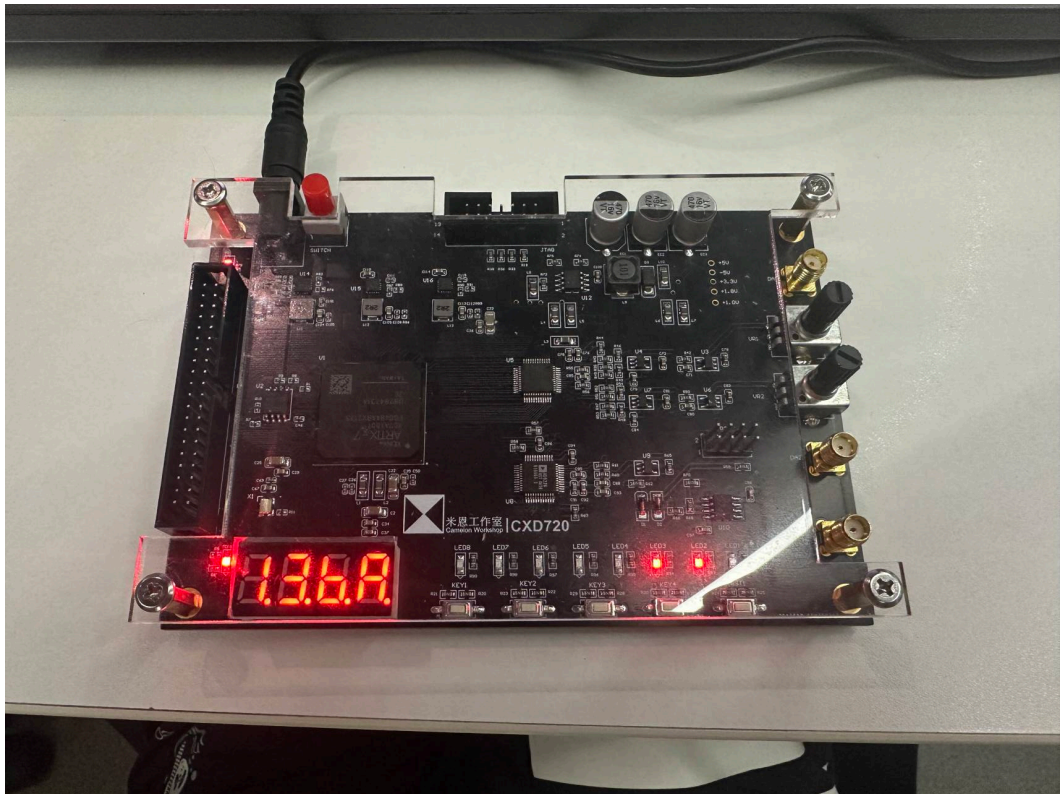


图 15 按下 KEY1 运行程序 1

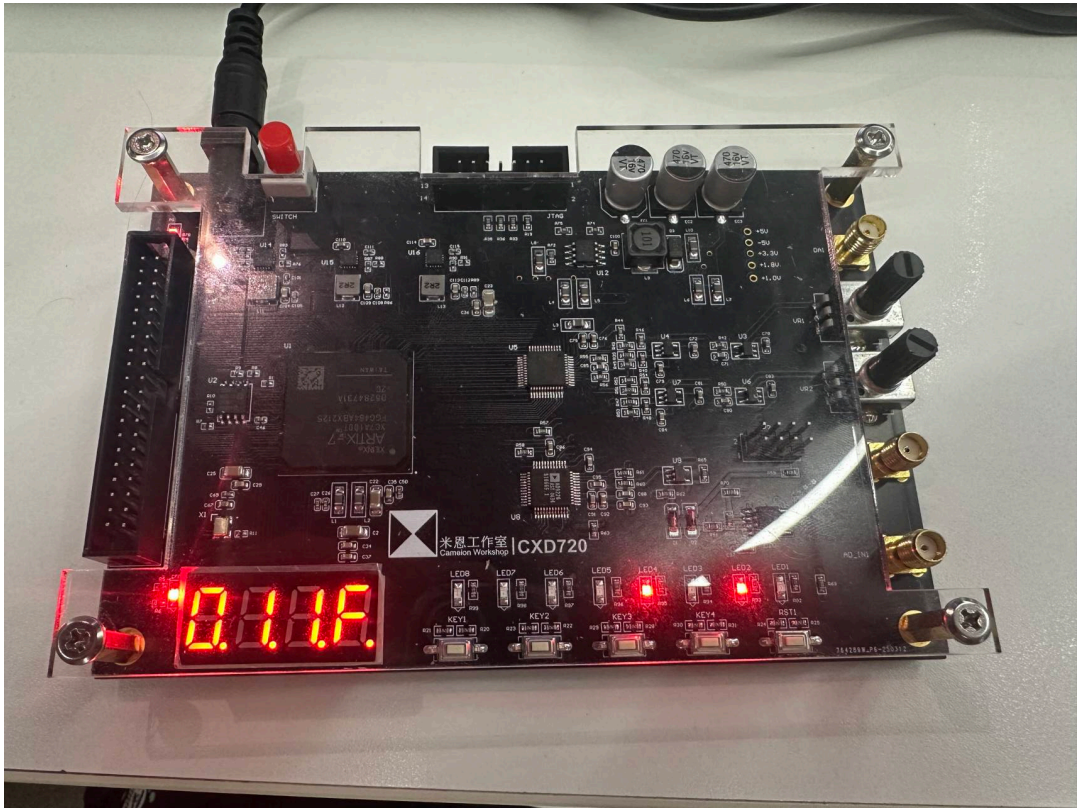


图 16 按下 KEY2 运行程序 2

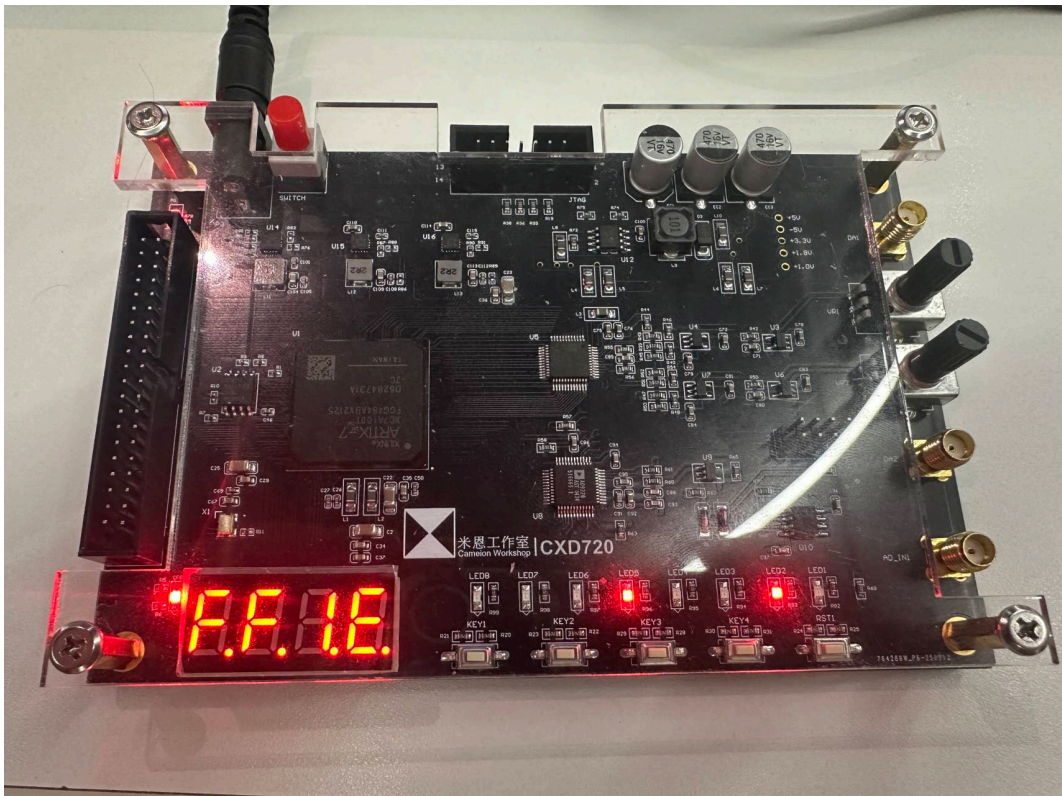


图 17 按下 KEY3 运行程序 3

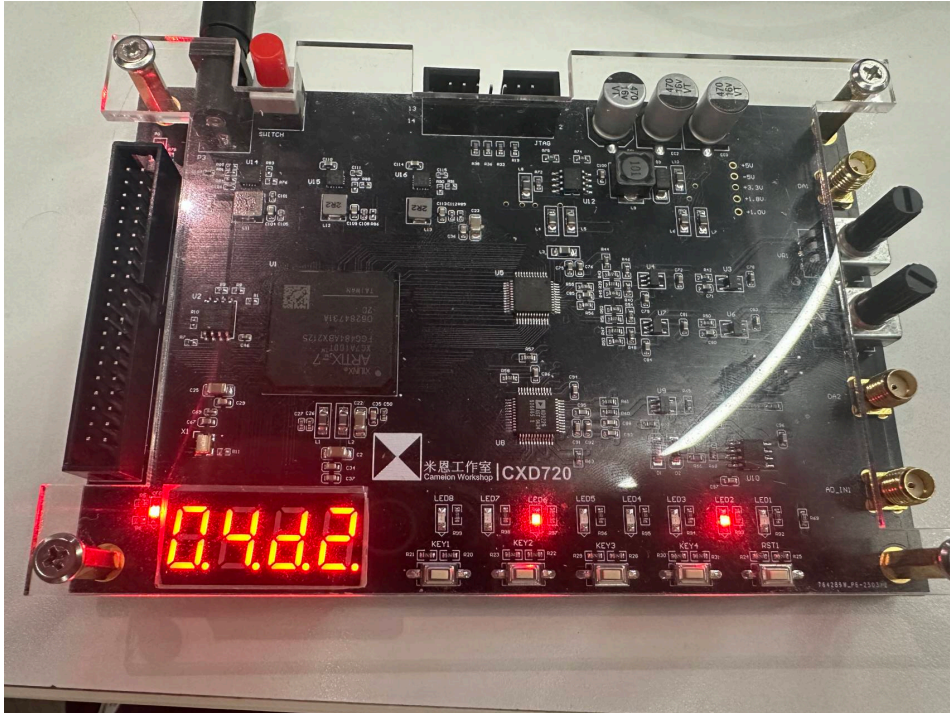


图 18 按下 KEY4 运行程序 4

运行程序	图片序号	运行结果（理想）	实际结果	判断
1	15	5050	0x13BA（5050）	正确
2	16	287	0x011F（287）	正确
3	17	65310	0xFF1E（65310）	正确
4	18	1234	0x04D2（1234）	正确

表 8 上板测试结果

可见最终上板测试结果也是正确的，因此完成上板验证。

七、总结与反思

本项目是 16 位 Verilog CPU 项目，支持在 Vivado 中进行行为级仿真、综合后时序仿真、布局布线后时序仿真，并可部署到 FPGA 开发板上运行。

本项目实现了：

- 微程序控制器
- 基于累加器的简化 CPU 数据通路
- 内置测试程序镜像的程序/数据存储器

- 板载按键选择并启动测试程序
- 通过数码管显示运行结果

本项目采用 16 位累加器型 CPU，8 位程序计数器和 8 位存储器地址空间，22 位微指令控制字，使用 CAR+ 控制存储器 ROM 的微程序控制流程，支持算术、逻辑、移位、访存和跳转指令，内置 4 组按键自检测测试程序，使用消抖按键 + 时钟使能。

在完成项目后，我发现该项目仍有进一步优化空间，我们重述图 5 如下：

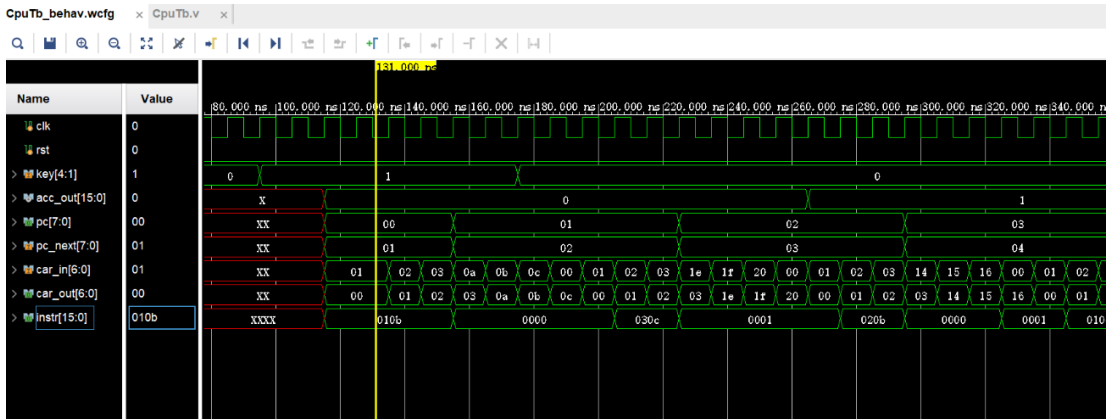


图 19 行为仿真的 CAR、PC

在图 5 中我们可以看到，我们以 PC 为 2 举例，在 PC 为 2 的这段时间内，前半时间在运行上一个 PC 取到的指令，后半时间在运行取指指令，显然取指操作和执行操作是不冲突的，因此我们可以将该程序修改成一个二阶流水线的 CPU，即将 FETCH 和 EXECUTE 操作分开，以下表的形式实现

	T1	T2	T3	T4	T5	T6	T7	T8	T9
I1	FI	EU							
I2		FI	EU						
I3			FI	EU					
I4				FI	EU				
I5					HALT				
I6						FI	EU		
I7							FI	EU	

表 9 预想流水线表现形式

若在此项目中引入流水线设计，则运行效率将提升

$$S = \frac{nk}{k+n-1} = \frac{2n}{n+1} \quad (2)$$

当 n 很大时，可认为 $S = 2$ ，即效率是原来的两倍，但是这样处理下可能会引入控制冒险、数据冒险、结构冒险等问题，需要额外处理，因此这里只提出一个想法，待实际编写验证。

八、附录

仿真用代码 CpuTb.v

```
`timescale 1ns / 1ps

module CpuTb;

    reg clk;
    reg rst;
    reg [4:1] key;
    integer cycle_count;

    wire [7:0] seg;
    wire [3:0] an;
    wire [8:1] led;

    CpuTop uut(
        .clk(clk),
        .rst(rst),
        .key(key),
        .seg_ap(seg),
        .seg_s(an),
        .led(led)
    );

    always #5 clk = ~clk;

    task press_reset;
        begin
            rst = 1'b1;
            repeat (8) @(posedge clk);
            rst = 1'b0;
            repeat (4) @(posedge clk);
        end
    end
```

```
endtask
```

```
task press_key;
    input integer idx;
    begin
        key[idx] = 1'b1;
        repeat (8) @(posedge clk);
        key[idx] = 1'b0;
    end
endtask
```

```
task wait_halt_and_check;
    input [15:0] expected;
    input [127:0] name;
    begin
        cycle_count = 0;
        while ((led[2] == 1'b1) && (cycle_count < 100)) begin
            @(posedge clk);
            cycle_count = cycle_count + 1;
        end

        cycle_count = 0;
        while ((led[2] != 1'b1) && (cycle_count < 12000)) begin
            @(posedge clk);
            cycle_count = cycle_count + 1;
        end

        if (led[2] != 1'b1) begin
            $display("=====");
            $display("TEST FAIL : %0s", name);
            $display("REASON      : TIMEOUT WAITING FOR HALT");
            $display("CYCLES       : %0d", cycle_count);
            $display("=====");
            $finish;
        end

        repeat (2) @(posedge clk);

        if (uut.acc_out == expected) begin
            $display("=====");
            $display("TEST PASS : %0s", name);
            $display("ACC        : %0d (0x%04h)", uut.acc_out, uut.acc_out);
            $display("=====");
        end else begin
```

```

        $display("=====");
        $display("TEST FAIL : %0s", name);
        $display("EXPECTED   : %0d (0x%04h)", expected, expected);
        $display("ACTUAL     : %0d (0x%04h)", uut.acc_out, uut.acc_out);
        $display("=====");
        $finish;
    end
end
endtask

initial begin
    $display("CPU_TB_SUITE_V2");
    clk = 1'b0;
    rst = 1'b0;
    key = 4'b0000;

    repeat (10) @(posedge clk);

    // key[1] : 1+...+100 = 5050
    press_key(1);
    wait_halt_and_check(16'd5050, "KEY1 SUM1TO100");

    // rst -> key[2] : MUL/AND/OR test = 287
    press_reset;
    press_key(2);
    wait_halt_and_check(16'd287, "KEY2 LOGIC_MUL");

    // rst -> key[3] : NOT/SHIFTR/SHIFTL test = 0xFF1E
    press_reset;
    press_key(3);
    wait_halt_and_check(16'hFF1E, "KEY3 SHIFT_NOT");

    // rst -> key[4] : JMPGEZ/JMP test = 1234
    press_reset;
    press_key(4);
    wait_halt_and_check(16'd1234, "KEY4 JUMP");

    $display("-----");
    $display("ALL TESTS FINISHED");
    $display("-----");
    $finish;
end
endmodule

```
